Chapter 1: Introduction to Reado.ai

1.1 Background and Context

The rapid proliferation of digital communication networks, electronic publishing platforms, and automated journalistic workflows has resulted in an exponential accumulation of unstructured textual data across the global information ecosystem. Readers, domain analysts, legal practitioners, and scientific researchers are routinely required to navigate voluminous corpuses of literature, including peer-reviewed manuscripts, regulatory filings, corporate white papers, and syndicated news streams. While the accessibility and democratization of digital documents have expanded significantly, human cognitive bandwidth, attention spans, and reading speeds have remained fundamentally bounded. This systemic divergence between information volume and human processing capacity leads directly to severe cognitive fatigue, superficial skim-reading practices, and an elevated probability of overlooking mission-critical nuances, technical qualifications, and evidentiary assertions within lengthy source texts.

Natural Language Processing (NLP) and Computational Linguistics have long treated automated document summarization as a foundational mechanism to alleviate this cognitive bottleneck. Historically, the summarization literature has bifurcated computational approaches into two primary operational paradigms: extractive summarization and abstractive summarization. Extractive frameworks approach the task as a combinatorial sequence-selection problem. Statistical and graph-theoretic algorithms—such as Term Frequency-Inverse Document Frequency (TF-IDF), TextRank, LexRank, and maximal marginal relevance scoring—assign numerical saliency weights to discrete sentence units extracted directly from the source material. Sentences exceeding a threshold score are assembled verbatim into an output digest. Although extractive pipelines inherently preserve grammatical correctness and prevent semantic drift, they consistently suffer from significant structural limitations. Because sentences are severed from their original discourse context, extracted outputs frequently exhibit severe discourse incoherence, dangling referents, ambiguous pronominal attachments, and unnecessary stylistic filler that degrades overall communicative efficiency.

In contrast, abstractive summarization mirrors human intellectual synthesis. Rather than performing simple sentence copying, an abstractive system reads the entire source context into continuous latent semantic vector representations, identifies latent conceptual hierarchies, and synthesizes entirely new sentences token-by-token. This approach enables dynamic paraphrasing, multi-sentence fusion, syntactic restructuring, and the systematic elimination of peripheral anecdotes. Driven by the advent of deep sequence-to-sequence neural architectures, recurrent encoder-decoder topologies, and attention-based Transformer models, abstractive text generation has emerged as the definitive standard for enterprise and academic document compression.

Despite these dramatic architectural breakthroughs, contemporary abstractive summarization deployments remain predominantly static and monolithic. Existing industrial application programming interfaces (APIs)

and pre-trained checkpoint baselines generate a single, unyielding summary of static length and uniform linguistic complexity for any given source document. Such static generation pipelines completely disregard user-specific cognitive constraints, variable operational environments, and targeted reading proficiencies. A postgraduate domain specialist investigating technical mechanics requires an analytical, structured synthesis that preserves technical nomenclature, quantitative figures, and causal linkages. Conversely, an executive or casual reader operating under extreme temporal limitations requires an immediate, plain-language digest stripped of recursive clauses and parenthetical discourse. A rigid, one-size-fits-all model inherently compromises across these divergent use cases, generating outputs that are either overly simplified for advanced research or unnecessarily dense for rapid decision-making. To overcome these limitations, modern natural language systems must transition toward controllable, adaptive text generation architectures.

1.2 Motivation: Digital Content Overload

Digital content overload is characterized by the structural imbalance between daily information acquisition demands and bounded biological reading thresholds. The modern digital workplace and academic landscape require continuous engagement with lengthy reports, policy documents, and dynamic media cycles. Consequently, knowledge workers spend a disproportionate share of productive hours filtering redundant descriptions rather than extracting actionable intelligence.

Standard digital interfaces provide passive reading environments. Readers are forced to manually alter their reading velocity or depend on fragmented scanning heuristics, which often leads to the omission of vital caveats, methodology conditions, and structural premises. Traditional computational summarization tools fail to resolve this condition due to three primary limitations:

1. *Agnosticism Toward Reader Context*: Commercial models process inputs uniformly, applying identical generation heuristics whether the intended recipient is a school student, a technical engineer, or an executive manager. The complete absence of parameter-level controllability leaves the consumer unable to modulate output density to match real-time reading goals.

2. *Fixed Compression Ratios*: Conventional generation pipelines apply static token budgets. They lack the architectural flexibility to transition dynamically from rapid, low-footprint distillations (under 40 words) to detailed analytical syntheses (100 to 200 words) without requiring the deployment and maintenance of distinct, disconnected neural pipelines.

3. *Monolithic Syntactic and Lexical Density*: Off-the-shelf generative checkpoints reflect the uniform lexical distributions of their underlying pre-training corpora (predominantly standard news wire copy). They are unable to synthesize plain, monosyllabic declarative statements on the one hand, or elevate stylistic vocabulary and complex multi-clause dependencies on the other.

These operational constraints motivate the design and implementation of Reado.ai—an intelligent, context-driven generative platform engineered to give readers explicit, interactive control over the verbosity, syntactic depth, and linguistic complexity of generated text summaries.

1.3 Problem Statement

The central engineering and algorithmic challenge addressed in this research can be formally defined as follows:

"How can an end-to-end natural language processing system be designed, calibrated, and deployed to dynamically generate contextually faithful, abstractive summaries conditioned across discrete complexity and verbosity tiers, while systematically mitigating generative hallucinations, circular degenerative repetitions, and operational inference latency within a responsive web architecture?"

This overarching research question decomposes into several core technical sub-challenges:

- *Controllable Discrete Generation*: Steering deep autoregressive sequence-to-sequence language models to strictly respect discrete token budgets and complexity constraints without collapsing syntactic fluency or corrupting grammatical dependencies.
- *Linguistic Tier Calibration*: Formulating reproducible decoding policies and adversarial loss constraints that govern vocabulary selection, clausal nesting, and information density across three distinct reader tiers (Easy, Hard, and Editorial).
- *Degenerative Mode Suppression*: Suppressing common generative artifacts inherent to autoregressive beam search decoding—specifically hallucinations, semantic drift, and circular n-gram repetition loops—during aggressive compression phases.
- *Full-Stack Systems Engineering*: Bridging the divide between high-parameter deep learning inference backends and client-side web interfaces to deliver low-latency, controllable text transformations within an intuitive user workflow**.**

1.4 Research Objective and Scopes

To address the defined problem statement, this dissertation pursues the following primary research and engineering objectives:

1. *Architect a Multi-Tier Controllable Text Generation Core*: Develop an abstractive summarization pipeline utilizing deep sequence-to-sequence Transformer architectures, introducing conditional decoding parameters and length-penalty regularizations to enforce explicit token constraints.
2. *Formulate and Calibrate the Tri-Level Complexity Architecture*: Establish three distinct, calibrated operational modes within a unified platform:

- Level 1 (Easy Mode - TL;DR): High-ratio semantic compression optimized for rapid cognitive assimilation, featuring short declarative syntax, reduced subordinate clause density, and everyday vocabulary.
- Level 2 (Hard Mode - Analytical): Intermediate compression calibrated for technical and academic consumption, retaining causal structures, quantitative data points, and domain dependencies within structured sentences.
- Level 3 (Editorial Mode - Publication-Grade): Moderate compression emphasizing stylistic depth, elevated lexicon, and comprehensive context retention suitable for professional publication standards.

3. *Eliminate Generative Pathologies*: Implement constrained decoding strategies—including beam search optimization, n-gram blocking mechanisms, and length penalties—to eliminate degenerate circular looping and minimize ungrounded factual hallucinations.
4. *Empirical Benchmarking and Metric Validation*: Quantify system performance using benchmark datasets through standard NLP evaluation suites, measuring lexical overlap (ROUGE-1, ROUGE-2, ROUGE-L), semantic fidelity, and readability metrics (Flesch Reading Ease, Flesch-Kincaid Grade Level) across all operational tiers.
5. *Develop and Integrate the Reado.ai Web Platform*: Engineer a production-ready, full-stack web application leveraging Next.js 15, Drizzle ORM, NeonDB serverless PostgreSQL, and an asynchronous GPU inference backend to provide users with real-time text input, dynamic tier switching, and persistent generation history.

Research Scope and Boundaries: The experimental boundaries of this project focus on English-language expository, academic, and journalistic prose. Input context lengths are bounded to 1,024 subword tokens, mapping directly to high-throughput neural inference environments optimized for consumer-grade GPU hardware.

1.5 Organization of the Report

The remainder of this dissertation is systematically organized into four major thematic parts across 17 chapters:

- *Part I: Foundations and Literature Review (Chapters 1–5)*: Introduces the core concepts of the Reado.ai platform, details the evolution from extractive heuristics to neural abstractive summarization, traces deep learning foundations from Recurrent Neural Networks to the Attention mechanism, analyzes Transformer model architectures, and reviews modern pre-trained sequence-to-sequence language models such as BART and T5.
- *Part II: Adversarial Formulation and Controllable Generation (Chapters 6–8)*: Presents the theoretical formulation of Generative Adversarial Networks for discrete text, establishes the mathematical and

algorithmic architecture of the Reado.ai Tri-Level Contextual Engine, and examines decoding strategies designed to mitigate degenerative loops and factual hallucinations.

- *Part III: Data Pipelines and Training Methodology (Chapters 9–11)*: Examines benchmark training corpora (CNN/DailyMail, XSum), explores subword tokenization strategies (Byte-Pair Encoding, SentencePiece), and documents the hardware setup, mixed-precision environments, and optimization schedules.
- *Part IV: Quantitative Evaluation and System Deployment (Chapters 12–17)*: Formulates linguistic readability metrics, presents empirical benchmark results and comparative tier evaluations, examines qualitative edge cases, outlines the software architecture of the Next.js 15 full-stack platform, and concludes with key contributions, limitations, and future research directions.

Chapter 2: Text Summarization and Generation Paradigms

2.1 The Spectrum of Text Summarization

Automated text summarization stands as one of the definitive benchmarks within Natural Language Processing (NLP) and Computational Linguistics. The primary objective is to take a source document $\mathcal{D}$ composed of a sequence of tokens or sentences and generate a concise surrogate text $\mathcal{S}$ that accurately retains the core factual propositions, thematic elements, and communicative intent of the original material while satisfying a designated length constraint $|\mathcal{S}| \ll |\mathcal{D}|$.

The theoretical spectrum of text summarization is traditionally structured around three core operational dimensions:

1. *Output Mechanics (Extractive vs. Abstractive)*: Extractive paradigms select, rank, and concatenate exact surface-level units directly from the source text. Abstractive paradigms encode the underlying semantic context into continuous latent representations and generate novel lexical tokens autoregressively.
2. *Contextual Scope (Single-Document vs. Multi-Document)*: Single-document summarization targets isolated narrative streams, scientific articles, or news reports. Multi-document summarization requires an algorithm to aggregate information across heterogeneous sources, resolve conflicting facts, eliminate cross-document redundancies, and synthesize a cohesive global perspective.
3. *User Conditioning (Generic vs. Query-Focused)*: Generic summarization produces an objective, generalized distillation of the primary themes contained in a text. Query-focused summarization conditions the condensation process on external search queries, user intents, or domain-specific constraints.

Within this taxonomic spectrum, contemporary research has migrated decisively from surface-level extractive selection toward deep abstractive generation. While extractive approaches provide hard guarantees against factual hallucinations, their inability to condense complex cross-sentence arguments creates fundamental barriers to readability and information density.

2.2 Extractive Summarization Techniques

Extractive summarization approaches the condensation task as a combinatorial sequence-selection problem. Rather than generating new natural language sentences, extractive systems evaluate individual sentences or clauses $s_i \in \mathcal{D}$ using mathematical, statistical, or structural importance metrics, subsequently extracting the top-k scoring units to construct the summary:

$$\mathcal{S}_{\text{ext}} = \arg \max_{\mathcal{S} \subset \mathcal{D}, |\mathcal{S}| \leq k} \sum_{s_i \in \mathcal{S}} \text{Score}(s_i) - \lambda \cdot \text{Redundancy}(\mathcal{S})$$

Historically, extractive methods evolved through several computational phases:

- *Frequency-Based and Statistical Models*: Early foundational systems, pioneered by Luhn and Edmundson, relied on lexical frequency distributions, positional weighting, and cue-word dictionaries. The integration of Term Frequency-Inverse Document Frequency (TF-IDF) allowed algorithms to weigh sentences based on the statistical distinctiveness of their constituent terms relative to a background corpus:

$$\text{TF-IDF}(w, d, \mathcal{C}) = \text{TF}(w, d) \times \log\left(\frac{|\mathcal{C}|}{|\{d' \in \mathcal{C} : w \in d'\}|}\right)$$

While computationally efficient, pure frequency models are blind to synonymy, polysemy, and grammatical dependencies.

- *Graph-Theoretic Centrality Algorithms*: Algorithms such as TextRank and LexRank conceptualized documents as continuous semantic networks. Sentences represent vertices V, and edge weights E_{ij} correspond to lexical or semantic similarity scores (such as cosine similarity over bag-of-words or TF-IDF vectors). Applying random-walk algorithms analogous to PageRank, the stationary probability distribution yields a centrality score for each sentence:

$$S(V_i) = (1 - d) + d \sum_{V_j \in \text{Adj}(V_i)} \frac{w_{ji}}{\sum_{V_k \in \text{Adj}(V_j)} w_{jk}} S(V_j)$$

Vertices with the highest convergence scores represent the most central informational nodes of the discourse.

- *Machine Learning and Feature Extraction*: Supervised extractive approaches treat the problem as a sequence-labeling binary classification task, where each sentence s_i receives a label $y_i \in \{0,1\}$. Support Vector Machines (SVMs), Naive Bayes classifiers, and early Recurrent Neural Networks (RNNs) evaluated feature vectors containing sentence length, relative paragraph position, presence of named entities, and overlap with document titles.

- *Neural Latent Selectors (BERTSUM)*: Modern extractive architectures utilize bidirectional Transformer encoders (such as BERT) fine-tuned with inter-sentence classification layers. By inserting specialized tokens (such as [CLS] tokens at the start of every sentence), the Transformer outputs a contextual sentence representation vector h_i, which passes through a sigmoid classification layer to predict extraction probabilities.

Structural Pathologies of Extractive Frameworks

Despite preserving factual accuracy by copying human-authored text verbatim, extractive summarization exhibits fundamental linguistic failure modes:

1. *Discourse Incoherence*: Sentences are extracted out of their native context, severing rhetorical transitions and resulting in disjointed, jarring reading flows.

2. *Dangling Co-References*: Extracted sentences frequently contain anaphoric pronouns ("he," "they," "this finding") without including the antecedent noun introduced in preceding, unselected sentences.

3. *Stylistic Redundancy and Low Compression Density*: Human writers naturally weave peripheral details into primary clauses. Extractive systems must take entire sentences, incorporating background filler, parenthetical qualifiers, and conversational anecdotes, thereby wasting limited word budgets on low-value context.

2.3 Abstractive Summarization Techniques

Abstractive summarization circumvents the structural rigidities of extractive selection by reframing summarization as a conditional language modeling and sequence-to-sequence translation task. Formally, given a source sequence $\mathrm{x} = (x_1, x_2, \ldots, x_M)$, the abstractive model parameters θ are trained to maximize the conditional log-likelihood of generating a target sequence $\mathrm{y} = (y_1, y_2, \ldots, y_N)$ where $N \ll M$:

$$\mathcal{L}(\theta) = \sum_{t=1}^{N} \log P\left(y_t \mid y_{<t}, \mathrm{x}; \theta\right)$$

Abstractive synthesis operates across several interconnected linguistic operations:

- *Paraphrasing and Sentence Fusion*: Condensing the core propositions of two or three sequential sentences into a single, cohesive, compound sentence.
- *Lexical Substitution*: Replacing complex domain explanations with concise, high-impact terminology or simpler vocabulary depending on target requirements.
- *Information Elimination*: Selectively pruning subordinate clauses, parenthetical remarks, and non-restrictive appositives while preserving the primary semantic predicate.

Evolution of Neural Sequence-to-Sequence Modeling

1. *Recurrent Encoder-Decoder Frameworks*: Early abstractive neural architectures utilized Long Short-Term Memory (LSTM) networks or Gated Recurrent Units (GRUs) to compress the input document into a single static hidden vector h_M, which a decoder network iteratively unrolled into output tokens. These models struggled severely with long-term dependencies, frequently suffering from catastrophic forgetting when processing inputs exceeding 100 words.

2. *Attention-Augmented Recurrent Models*: Bahdanau and Luong introduced dynamic attention mechanisms that allowed the decoder to compute alignment weights α_{ti} over all encoder hidden states at each decoding step t. This innovation enabled the network to dynamically focus on relevant sections of the source document during text synthesis.

3. *The Pointer-Generator Architecture*: To resolve the "out-of-vocabulary" (OOV) problem and combat factual hallucinations, See et al. introduced the Pointer-Generator network. The architecture computes

a generation probability $p_{\text{gen}} \in [0,1]$ at each time step, allowing the model to smoothly interpolate between generating a token from the vocabulary distribution P_{vocab} and copying a token directly from the source text via pointing:

$$P(w) = p_{\text{gen}} P_{\text{vocab}}(w) + (1 - p_{\text{gen}}) \sum_{i:w_i=w} a_i^t$$

This hybrid approach served as a foundational bridge between extractive fidelity and abstractive fluency.

4. *Pre-trained Denoising Autoencoders*: Modern abstractive summarization relies on large-scale self-supervised sequence-to-sequence Transformers, such as BART (Bidirectional and Auto-Regressive Transformers) and T5 (Text-to-Text Transfer Transformer). Pre-trained on multi-gigabyte textual corpora using corruptive noise functions (token masking, sentence deletion, document rotation, and text infilling), these models possess rich linguistic representations, world knowledge, and fluent generative capabilities.

2.4 Limitations of Conventional Fixed-Length Summaries

Despite the high syntactic fluency achieved by contemporary pre-trained language models, production-grade summarization deployments remain predominantly static. Commercial APIs and baseline systems typically generate a single summary of arbitrary, fixed length for any given document.

This static paradigm imposes several operational limitations:

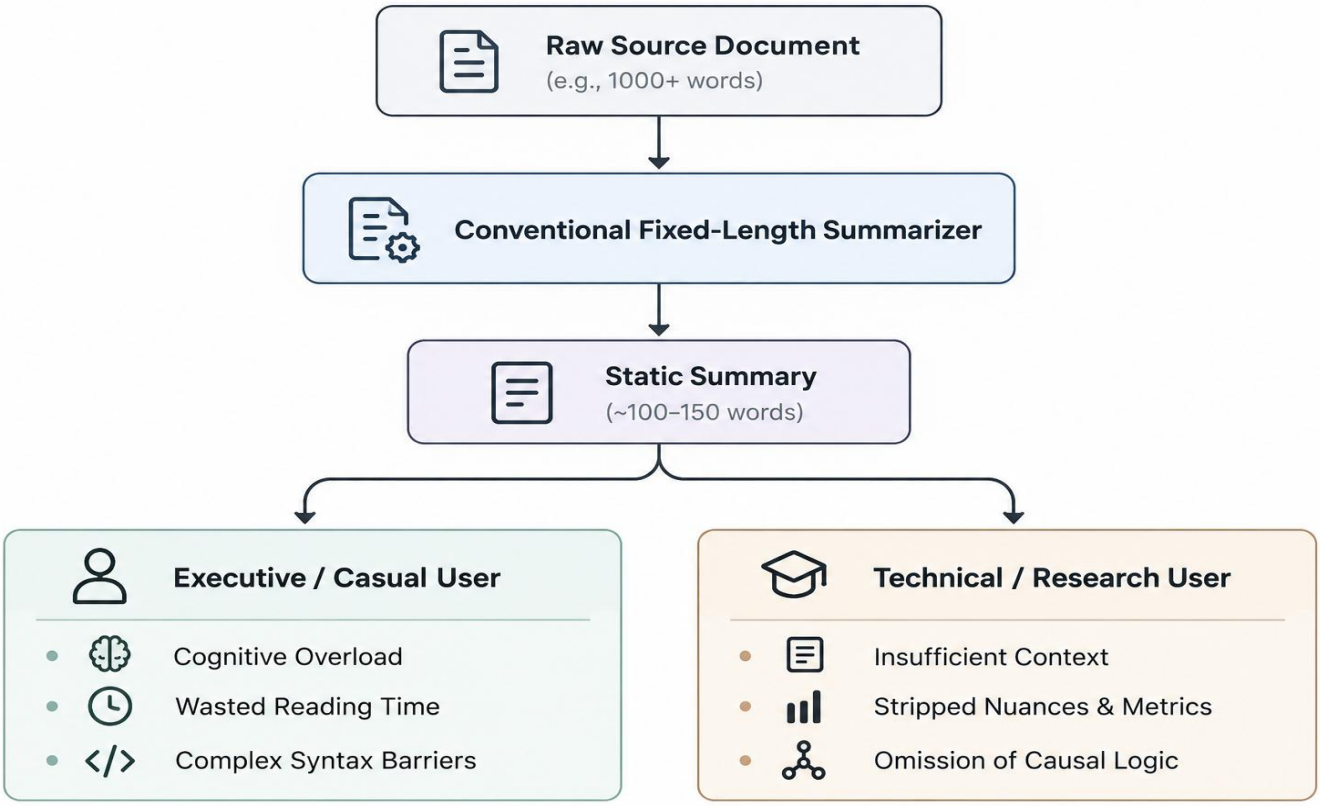


Figure 2.1: Limitations of Conventional Fixed-Length Summarization Pipelines

Agnosticism Toward Cognitive Bandwidth: Readers approach documents under fundamentally distinct constraints. A mobile user scanning news headlines requires an immediate, high-compression brief of under 30 words. An analyst or researcher digesting an experimental paper requires a 150-word synthesis that systematically outlines methodology, numerical metrics, and qualifications. A fixed-length output inevitably fails both users: it provides too much superficial detail for the former and discards essential evidentiary depth for the latter.

Exposure Bias and Degenerative Mode Collapse: Fixed-length generation models trained via standard maximum likelihood estimation (MLE) suffer from exposure bias. During autoregressive inference, small early decoding errors compound down the generation chain. If an unconditioned model is pushed to generate longer sequences than its standard checkpoint calibration, it regularly defaults to repetitive phrase looping, semantic drift, or hallucinations.

Monolithic Lexical Density: Standard summarization checkpoints reflect the uniform lexical and syntactic distributions of their primary training benchmarks (such as news wire articles). They lack the structural agility to shift between plain, low-Flesch-Kincaid monosyllabic constructions for non-specialists and sophisticated, multi-clause syntactic structures for academic or editorial dissemination.

2.5 The Need for Dynamic, Controllable Generation

To resolve the structural shortcomings of static summarization, modern NLP must advance toward **Controllable Text Generation (CTG)**. Controllable summarization decouples the synthesis process from static decoding policies, enabling users to steer the generation trajectory across specific dimensions: length, style, vocabulary complexity, and information density.

Dynamic controllability transforms the summarization system from a rigid black-box pipeline into an interactive cognitive tool:

- *Parameter-Driven Length and Verbosity Control*: Rather than accepting arbitrary stopping criteria dictated by the end-of-sequence (</s>) token, the decoding architecture allows users to modulate minimum and maximum length bounds alongside length-penalty parameters (α):

$$S(Y, X) = \frac{\log P\,(Y \mid X)}{(5 + |Y|)^{\alpha}/(5+1)^{\alpha}}$$

By adjusting α, the decoding beam can aggressively prioritize dense semantic compression or incentivize prolonged, comprehensive context retention.

- *Lexical and Syntactic Complexity Modulation*: Controllable generation empowers the underlying model to adapt its vocabulary and syntactic structures to match the target reading audience. Synthesizing summaries across discrete reading levels—ranging from elementary syntactical

structures to publication-grade editorial prose—demands systematic control over vocabulary distributions, subordinate clause depth, and grammatical complexity.

- *Operational Architecture of Reado.ai*: Reado.ai addresses this imperative through its tri-level complexity architecture, categorizing controllable generation into three calibrated operational modes:

  i. **Easy (TL;DR):** High-ratio semantic compression (>85%) optimized for rapid assimilation using simplified, direct declarative phrasing.

  ii. **Hard (Analytical):** Intermediate compression (70%–80%) designed for technical and academic consumption, retaining multi-clause dependencies, numerical metrics, and structural arguments.

  iii. **Editorial (Publication-Grade):** Moderate compression (50%–60%) tailored for professional dissemination, maintaining stylistic richness, narrative continuity, and sophisticated vocabulary.

Chapter 3: Neural Networks and Deep Learning in NLP

3.1 Recurrent Neural Networks (RNNs) and LSTMs

Prior to the advent of attention-centric neural modeling, automated natural language processing and sequence transduction tasks relied predominantly on Recurrent Neural Networks (RNNs). Unlike feed-forward networks that treat input features as statistically independent entities, an RNN is explicitly formulated to process sequential data $\mathbf{x} = (x_1, x_2, \ldots, x_T)$ by maintaining an internal continuous hidden state vector $\mathbf{h}_t \in \mathbb{R}^d$. At each discrete temporal step t, the recurrent transition function updates this internal state by combining the current input vector with the activation vector from the preceding time step:

$$\mathbf{h}_t = \tanh(\mathbf{W}_{hh}\mathbf{h}_{t-1} + \mathbf{W}_{xh}\mathbf{x}_t + \mathbf{b}_h)$$

Where $\mathbf{W}_{hh} \in \mathbb{R}^{d \times d}$ denotes the hidden-to-hidden recurrent transition matrix, $\mathbf{W}_{xh} \in \mathbb{R}^{d \times d_{in}}$ represents the input-to-hidden weight projection matrix, and $\mathbf{b}_h$ is the corresponding bias term.

The Problem of Exploding and Vanishing Gradients

Training standard recurrent networks requires backpropagation through time (BPTT). Computing the gradient of the scalar loss $\mathcal{L}$ with respect to the recurrent parameter matrix $\mathbf{W}_{hh}$ necessitates computing the product of Jacobian matrices across temporal sequences:

$$\frac{\partial \mathcal{L}}{\partial \mathbf{W}_{hh}} = \sum_{t=1}^{T} \sum_{k=1}^{t} \frac{\partial \mathcal{L}_t}{\partial \mathbf{h}_t} \left(\prod_{j=k+1}^{t} \frac{\partial \mathbf{h}_j}{\partial \mathbf{h}_{j-1}} \right) \frac{\partial \mathbf{h}_k}{\partial \mathbf{W}_{hh}}$$

As the temporal distance $t - k$ increases, the continuous matrix product $\prod_{j=k+1}^{t} \frac{\partial \mathbf{h}_j}{\partial \mathbf{h}_{j-1}}$ causes gradient values to scale exponentially according to the spectral radius of the transition matrix. If the largest eigenvalue of $\mathbf{W}_{hh}$ is strictly less than unity, gradients vanish exponentially toward zero, preventing the network from updating weights based on early contextual cues. Conversely, if the eigenvalue exceeds unity, gradients explode, destabilizing weight convergence and precipitating numerical overflow.

Long Short-Term Memory (LSTM) Networks

To counteract the vanishing gradient pathology, Hochreiter and Schmidhuber formulated the Long Short-Term Memory (LSTM) cell architecture. LSTMs introduce an additive internal memory conduit called the cell state ($\mathbf{c}_t$), regulated by three multiplicative gating structures parameterizing information flow:

- *Forget Gate:* Determines what proportion of the historical cell state should be discarded:

$$\mathbf{f}_t = \sigma\big(\mathbf{W}_f[\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_f\big)$$

- *Input Gate and Candidate State:* Determines what novel information should be assimilated into the internal memory state:

$$\mathbf{i}_t = \sigma(\mathbf{W}_i[\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_i)$$

$$\tilde{\mathbf{c}}_t = \tanh(\mathbf{W}_c[\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_c)$$

- *Cell State Update:* Executes additive linear state updating, preserving uninterrupted gradient backpropagation conduits across long sequence spans:

$$\mathbf{c}_t = \mathbf{f}_t \odot \mathbf{c}_{t-1} + \mathbf{i}_t \odot \tilde{\mathbf{c}}_t$$

- *Output Gate and Hidden Activation:* Emits the revised hidden representation passed to subsequent operational layers:

$$\mathbf{o}_t = \sigma(\mathbf{W}_o[\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_o)$$

$$\mathbf{h}_t = \mathbf{o}_t \odot \tanh(\mathbf{c}_t)$$

Where $\sigma(\cdot)$ denotes the element-wise logistic sigmoid activation, and $\odot$ represents the Hadamard element-wise vector product. While LSTMs extended memory horizons relative to standard vanilla RNNs, their sequential processing dynamics continued to present fundamental computational constraints during large-scale document modeling.

3.2 Sequence-to-Sequence (Seq2Seq) Frameworks

Sutskever et al. and Cho et al. formalized the Sequence-to-Sequence (Seq2Seq) neural architecture, enabling neural systems to map an input sequence of variable length $\mathbf{x} = (x_1, x_2, \ldots, x_M)$ to an output sequence of independent length $\mathbf{y} = (y_1, y_2, \ldots, y_N)$. This paradigm serves as the foundational architectural blueprint for neural machine translation, abstractive summarization, and conditional text generation.

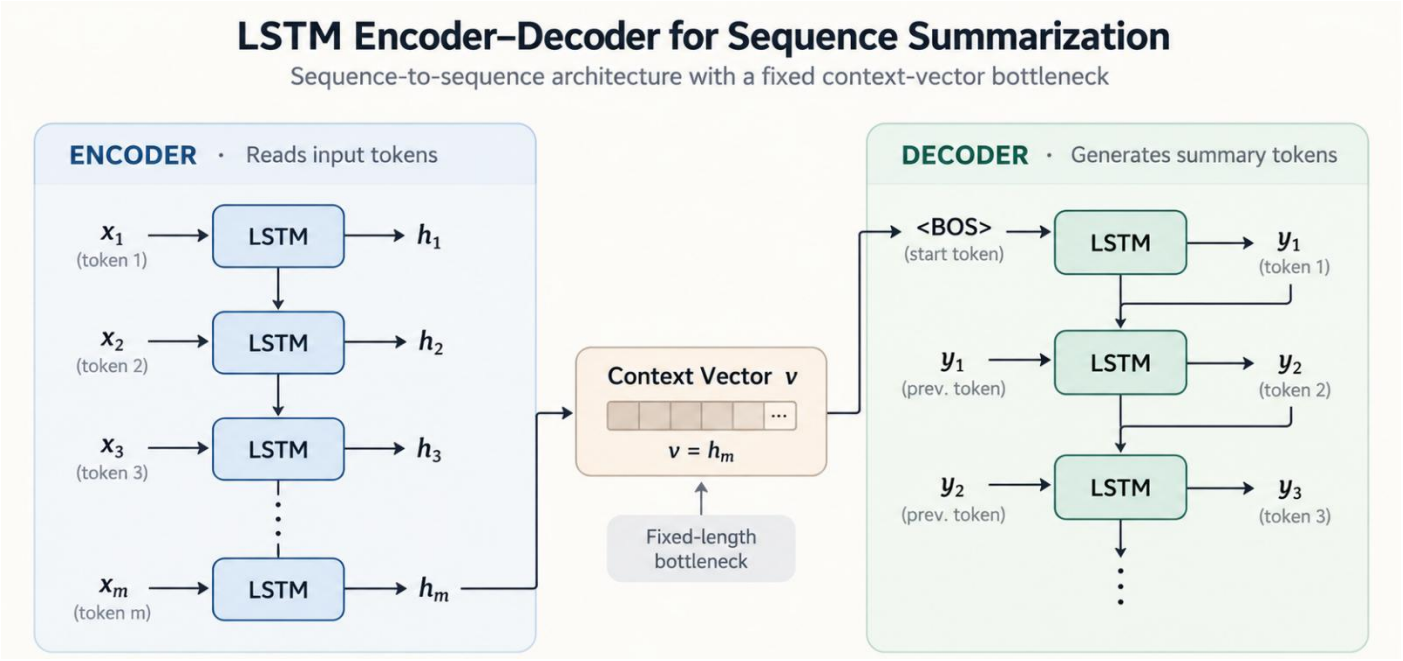


Fig 3.1 LSTM Encoder-Decoder for Seq2Seq Summarization

The canonical Seq2Seq framework comprises two asymmetric neural networks:

3.2.1 The Recurrent Encoder

The encoder sequentially consumes input tokens $x_m \in \mathbf{x}$. At each step, it updates its recurrent hidden state vector $\mathbf{h}_m$. Once the terminal token x_M is processed, the encoder emits its final hidden activation vector $\mathbf{h}_M$. This terminal vector is designated as the global context vector $\mathbf{v}$:

$$\mathbf{v} = \mathbf{h}_M = \text{EncoderRNN}(x_1, x_2, \dots, x_M)$$

The vector $\mathbf{v}$ is presumed to act as a complete, dense, fixed-dimensional semantic summary capturing the entirety of the input document's thematic context.

3.2.2 The Autoregressive Decoder

The decoder is a conditioned language generation model initialized using the context vector emitted by the encoder:

$$\mathbf{s}_0 = \mathbf{v}$$

At each output generation step t, the decoder maintains an internal hidden state $\mathbf{s}_t = \text{DecoderRNN}(\mathbf{s}_{t-1}, y_{t-1}, \mathbf{v})$. The probability distribution over the target vocabulary space $\mathcal{V}$ is derived by passing the hidden state through a linear projection layer parameterized by matrix $\mathbf{W}_s \in \mathbb{R}^{|\mathcal{V}| \times d}$ and normalizing via the Softmax function:

$$P(y_t \mid y_{<t}, \mathbf{x}) = \text{softmax}(\mathbf{W}_s \mathbf{s}_t + \mathbf{b}_s)$$

Training is conducted using teacher-forcing, where ground-truth tokens y^*_{t-1} are provided as inputs at each time step to maximize the cumulative sequence log-likelihood objective:

$$\mathcal{L}_{\text{Seq2Seq}}(\theta) = -\sum_{t=1}^{N} \log P\left(y_t^* \mid y_{<t}^*, \mathbf{x}; \theta\right)$$

3.3 The Information Bottleneck Problem

The primary theoretical and practical limitation of the classical Seq2Seq architecture is the Information Bottleneck Problem. In standard formulations, the model requires the encoder to compress an input document of arbitrary length, structural complexity, and conceptual density into a single, fixed-dimensional hidden vector $\mathbf{v} \in \mathbb{R}^d$.

This structural compression creates critical operational failures when applied to long-form document summarization:

1. *Capacity Saturation and Catastrophic Forgetting*: Vector spaces of fixed dimension d (typically $d \in [256,1024]$) possess finite informational entropy capacity. As source text length scales beyond 50 to

100 tokens, the continuous recurrent updates overwrite early network activations. The model experiences recency bias, preserving information situated near the end of the document while losing foundational premises introduced in early paragraphs.

2. *Loss of Fine-Grained Evidentiary Detail*: High-precision data points—such as numerical values, named entities, technical parameters, and conditional caveats—are lost during global vector compression. Consequently, the decoder is forced to hallucinate missing factual links or generate generic, non-committal phrasing during autoregressive decoding.

3. *Impeded Gradient Propagation*: During backpropagation through time, gradient signals emitted from the decoder must travel through the single context vector bridge $\mathbf{v} = \mathbf{h}_M$ before reaching early encoder states $\mathbf{h}_1, \ldots, \mathbf{h}_k$. This structural bottleneck compounds the gradient attenuation problem, hindering convergence across deep encoder networks.

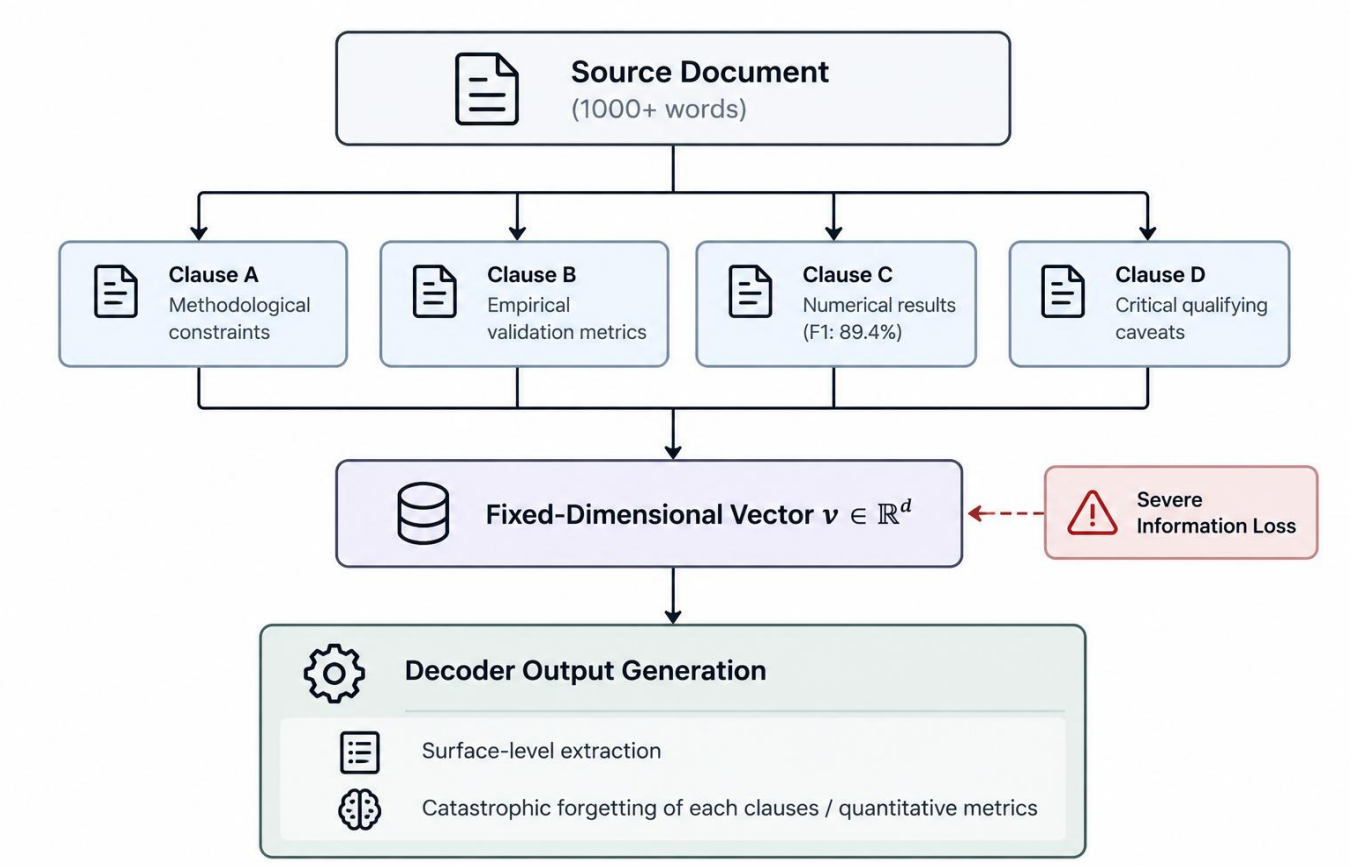


Fig 3.2 Information Loss in the Seq2Seq Encoder–Decoder Architecture

3.4 Additive Attention (Bahdanau Architecture)

Bahdanau et al. formulated additive attention by passing the current decoder hidden state $\mathbf{s}_{t-1}$ and each source encoder state $\mathbf{h}_i$ through a learned feed-forward neural layer with a hyperbolic tangent non-linearity:

$$e_{ti} = \mathbf{v}_a^\top \tanh(\mathbf{W}_a \mathbf{s}_{t-1} + \mathbf{U}_a \mathbf{h}_i)$$

Where $\mathbf{W}_a \in \mathbb{R}^{k \times d}$, $\mathbf{U}_a \in \mathbb{R}^{k \times d}$, and $\mathbf{v}_a \in \mathbb{R}^k$ are learned projection parameters. The scalar energy scores e_{ti} represent the raw semantic compatibility between the target decoding position and the source token at index i.

The alignment coefficients α_{ti} are obtained by normalizing the energy scores over the entire source document length using the Softmax function:

$$\alpha_{ti} = \frac{\exp(e_{ti})}{\sum_{k=1}^{M} \exp(e_{tk})}$$

The dynamic context vector $\mathbf{c}_t$ is computed as the probability-weighted expectation over all encoder hidden states:

$$\mathbf{c}_t = \sum_{i=1}^{M} \alpha_{ti} \, \mathbf{h}_i$$

The context vector $\mathbf{c}_t$ is then concatenated with the decoder hidden state $\mathbf{s}_t$ to derive the attentional hidden state $\tilde{\mathbf{s}}_t$, which directly informs the final output token distribution:

$$\tilde{\mathbf{s}}_t = \tanh(\mathbf{W}_c[\mathbf{c}_t; \mathbf{s}_t])$$

$$P(y_t \mid y_{<t}, \mathbf{x}) = \text{softmax}(\mathbf{W}_s \tilde{\mathbf{s}}_t)$$

3.4.2 Multiplicative Attention (Luong Architecture)

Luong et al. simplified the alignment scoring function by proposing multiplicative (dot-product) formulations, eliminating the intermediate feed-forward non-linearity in favor of direct matrix multiplication:

$$\text{Score}(\mathbf{s}_t, \mathbf{h}_i) = \begin{cases} \mathbf{s}_t^\top \mathbf{h}_i & \text{(Dot Product)} \\ \mathbf{s}_t^\top \mathbf{W}_a \mathbf{h}_i & \text{(General Bilinear Multiplicative)} \end{cases}$$

The multiplicative approach provides distinct computational advantages over additive scoring:

- **Matrix Multiplication Efficiency:** Multiplicative scoring can be formulated entirely as matrix multiplications ($\mathbf{S}\mathbf{W}_a\mathbf{H}^\top$), allowing implementations to exploit optimized Basic Linear Algebra Subprograms (BLAS) routines on GPU tensor cores.
- **Direct Gradient Paths:** Removing the hyperbolic tangent activation layer provides a direct linear gradient pathway from the output prediction loss back to intermediate encoder representations, accelerating model convergence.

The development of additive and multiplicative attention mechanisms fundamentally decoupled sequence generation from static recurrent memory limits. This architectural breakthrough laid the theoretical foundation for self-attention and the Transformer architectures analyzed in subsequent chapters.

Chapter 4: Transformer Architecture Overview

4.1 Transformer Model Architecture

The introduction of the Transformer model by Vaswani et al. replaced traditional recurrent neural networks with an architecture built entirely on self-attention mechanisms. By eliminating sequential recurrent processing, the model handles input sequences in parallel, dramatically improving training throughput on GPU accelerators and resolving the long-range dependency problem in NLP tasks.

The canonical Transformer relies on an asymmetric encoder-decoder topology. The encoder processes the source text bidirectionally to create contextualized vector representations, while the autoregressive decoder consumes these representations alongside previously generated tokens to predict the summary one token at a time.

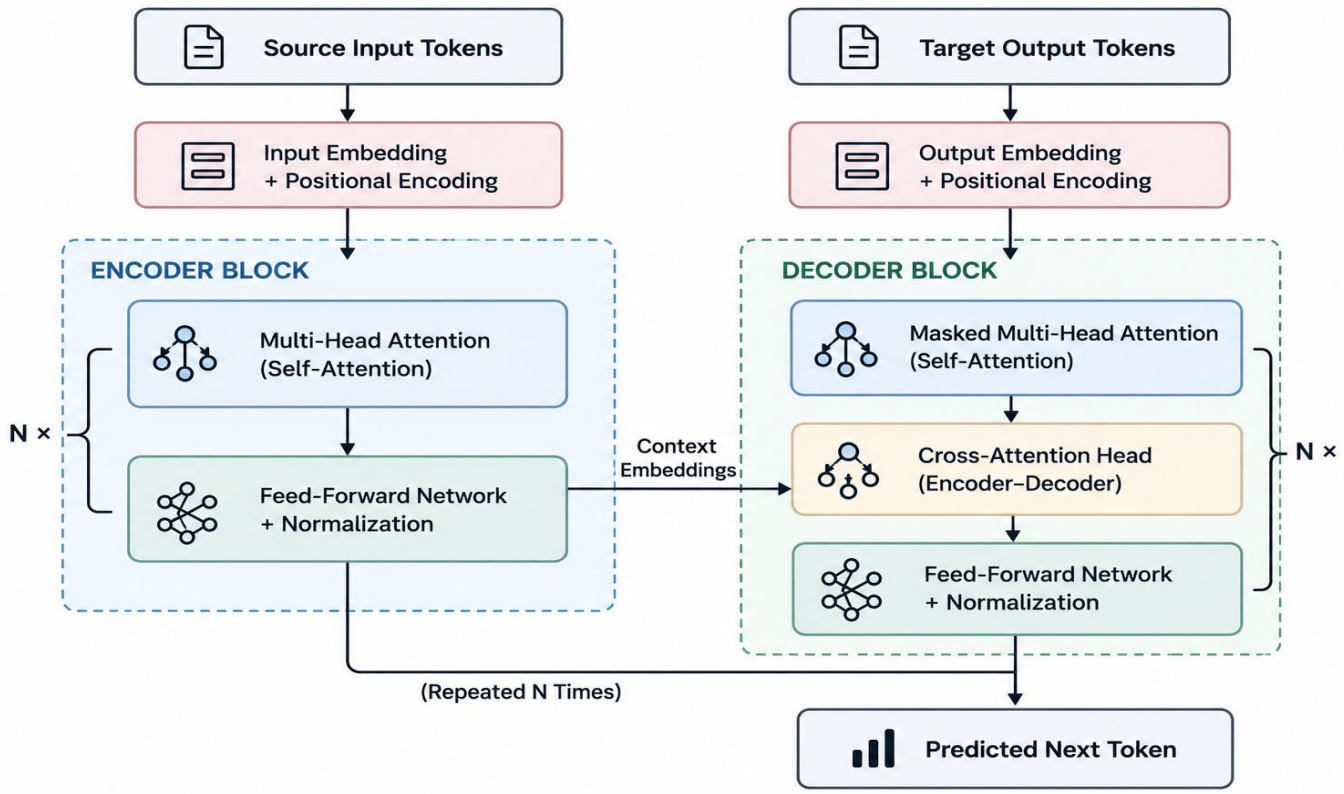


Fig 4.1 Transformer Encoder-Decoder Architecture

4.2 Scaled-Dot Product Attention

Scaled dot-product attention computes content-based alignment using three conceptual vectors: Queries (Q), Keys (K), and Values (V).

- Query (Q): The current token seeking context.
- Key (K): Candidate tokens offering contextual anchors.
- Value (V): The underlying semantic information stored across those tokens.

The dot product of Q and K measures semantic similarity. To prevent large vector dimensions from driving the softmax function into regions with near-zero gradients, the dot product is divided by a scaling factor proportional to the vector dimensionality ($\sqrt{d_k}$). The softmax distribution then weights the Value vectors to form the final contextual output.

Component	**Operational Role**	**Vector Representation**
Query (Q)	Probes the sequence to identify relevant semantic dependencies.	Current token embedding projection
Key (K)	Acts as an addressable index against which queries match.	Document token representation
Value (V)	The actual semantic payload retrieved during attention.	Unmasked information content
Scaling Factor ($\sqrt{d_k}$)	Stabilizes gradients by dampening extreme dot products.	Scalar dimensionality constant

Table 4.1 Attention Parameter in Scaled-Dot Product

4.3 Multi-Head Attention Mechanisms

A single attention operation risks averaging multiple distinct grammatical and semantic signals across the input. The Transformer addresses this by introducing Multi-Head Attention, which splits the hidden representations into parallel subspaces.

Each head projects Q, K, and V independently, allowing the network to capture distinct contextual patterns simultaneously:

- Syntactic Dependencies: Tracking local grammatical patterns, such as subject-verb agreement.
- Coreference Resolution: Matching pronouns to earlier nouns across long distances.
- Global Thematic Alignment: Evaluating high-level relationships across paragraph boundaries.

The outputs from all individual attention heads are concatenated and linearly projected back to the primary model dimension, producing a unified representation.

4.4 Positional Encodings & Feed-Forward Sublayers

Because self-attention is naturally order-agnostic, explicit positional information is required to preserve syntax, order, and reading flow.

4.4.1 Positional Encodings

The Transformer injects positional information by adding distinct frequency patterns to the initial token embeddings. Sine and cosine functions map coordinates across varying wavelengths, allowing the network to distinguish whether a word appears at the start, middle, or end of a sequence without using recurrence.

4.4.2 Feed-Forward Sublayers

Following the attention calculations, each Transformer layer includes a position-wise feed-forward network applied to each token position independently and identically. This module expands the representation space before projecting it back, deepening the network's capacity to synthesize abstract features. Each sublayer is surrounded by a residual connection followed by layer normalization, which stabilizes gradient propagation throughout deep network stacks.

4.5 Encoder v/s Decoder Representation

The Transformer partitions sequence transduction between two specialized structural components: the encoder stack and the decoder stack.

Structural Metric	**Encoder Block**	**Decoder Block**
Attention Scope	Full Bidirectional (Tokens see past and future)	Causal Unidirectional (Tokens only see the past)
Masking Policy	Padding mask only	Padding mask + Future lookahead mask
Sublayers per Block	2 (Self-Attention + Feed-Forward)	3 (Masked Self-Attn + Cross-Attn + Feed-Forward)
Functional Role	Encodes input document context	Generates summary tokens step-by-step
Cross-Attention Role	Generates Key (K) and Value (V) sources	Submits Query (Q) against encoder representations

Table 4.2 Difference between Representations of Encoder and Decoder

Chapter 5: Pre-Trainer Language Model for Text Generation

5.1 The Pre-training and Fine-tuning Paradigm

The pre-training and fine-tuning paradigm represents a fundamental shift in modern Natural Language Processing. Historically, neural architectures were trained from scratch on task-specific, human-annotated datasets. This classical supervised approach was severely constrained by the high cost and scarcity of labeled data, frequently causing deep networks to overfit when applied to complex sequence transduction tasks such as abstractive summarization.

The modern paradigm decouples representation learning into two distinct phases. During self-supervised pre-training, a deep neural model is exposed to vast, not curated textual corpora spanning hundreds of gigabytes, such as digitized books, news archives, and web crawls. Instead of relying on manual labels, the network learns the foundational mechanics of human language—including syntactic dependencies, semantic associations, world facts, and long-range contextual cues—by solving self-supervised surrogate tasks, such as predicting masked tokens or reconstructing corrupted text spans.

Once pre-training establishes a generalized language foundation, the model enters the fine-tuning phase. In this step, the network weights are initialized with the pre-trained parameters and adjusted using a smaller, task-specific dataset. For abstractive summarization, the model is supervised on document-summary pairs (such as CNN/DailyMail or XSum) using sequence cross-entropy loss. Fine-tuning aligns the model's rich latent linguistic representations with specific operational requirements, significantly reducing convergence time, improving factual fidelity, and yielding higher ROUGE scores compared to models trained from scratch.

5.2 Encoder-Decoder Models: T5 and BART

Sequence-to-sequence abstractive summarization requires an architecture capable of deeply understanding an arbitrary input passage while simultaneously generating a fluent, grammatically sound compression. Pre-trained encoder-decoder models are specifically engineered for this dual challenge, combining the contextual comprehension of bidirectional encoders with the generative fluency of autoregressive decoders.

The Text-to-Text Transfer Transformer (T5), developed by Raffel et al., frames every natural language processing task into a unified string-to-string format. Whether the task is classification, translation, question answering, or document summarization, T5 consumes a natural language task prefix (such as "summarize: ") alongside the source passage and generates the target text. T5 is pre-trained on the Colossal Clean Crawled Corpus (C4) using a span-corruption objective, where consecutive sequences of input tokens are masked with unique sentinel tokens and the decoder learns to reconstruct the missing spans. This text-to-text formulation provides architectural consistency and allows seamless transfer learning across disparate language tasks.

Bidirectional and Auto-Regressive Transformers (BART), formulated by Lewis et al., combines the bidirectional encoding capabilities of BERT with the autoregressive decoding structure of GPT. BART is

explicitly designed as a denoising autoencoder. During pre-training, input texts are corrupted using a diverse suite of noise transformations, including random token masking, token deletion, sentence permutation, document rotation, and text infilling via Poisson-distributed span lengths. The bidirectional encoder processes the corrupted document, while the causal autoregressive decoder learns to reconstruct the original, uncorrupted text. This denoising objective makes BART effective for abstractive summarization, as the model naturally learns to correct distorted syntactic flow, condense redundant clauses, and reorder information into coherent narrative summaries. Reado.ai leverages the "*facebook/bart-large-cnn"* checkpoint as its generative core due to its proven abstractive fidelity and balanced inference footprint.

Model	**Architectural Paradigm**	**Pre-training Objective**	**Primary Strength in Summarization**
BART	Sequence-to-Sequence (Bidirectional Encoder + Causal Decoder)	Denoising Autoencoding (Token masking, span infilling, sentence shuffling)	Exceptional structural paraphrasing and sentence reconstruction
T5	Sequence-to-Sequence (Unified Text-to-Text)	Span Corruption (Reconstructing missing multi-token spans marked by sentinels)	Multi-task versatility and cross-domain zero-shot adaptability
GPT-Style	Decoder-Only (Causal Autoregressive)	Next-Token Prediction (Unidirectional causal language modeling)	High conversational fluency and open-ended generative versatility

Table 5.1 Difference between T5 and BART Encoder-Decoder Models

5.3 Casual Language Models: GPT Architecture

Causal language models, epitomized by the Generative Pre-trained Transformer (GPT) series developed by Radford, Brown, et al., rely exclusively on a decoder-only architecture. Unlike encoder-decoder systems that dedicate separate parameters to document comprehension and sequence generation, GPT models process all text sequentially through stacked masked self-attention blocks. Every token can only attend to its historical left-hand context, completely preventing bidirectional cross-token interaction during initial document ingestion.Pre-training for causal language models is governed entirely by standard autoregressive next-token prediction over immense textual corpuses. Given an input context sequence, the model maximizes the likelihood of predicting the immediate subsequent token across billions of parameters. At inference time, these

architectures perform summarization through prompt engineering and in-context learning. By prepending a instructional prompt (such as "Summarize the following text in three concise bullet points:"), the causal decoder continues generating text, shifting from prompt consumption directly into summary generation.

While large causal language models demonstrate impressive zero-shot fluency and contextual adaptability, they present distinct operational trade-offs when deployed within dedicated document compression pipelines. Because causal decoders process source documents unidirectionally, they cannot build fully contextualized, bidirectional representations of early paragraphs informed by concluding sections. Furthermore, their immense parameter scale introduces substantial computational overhead, high GPU memory utilization, and increased inference latency compared to dedicated, task-optimized sequence-to-sequence models like BART.

5.4 Domain Adaptation and Parameter Efficiency

Deploying large pre-trained language models across specialized domains - such as legal contracts, biomedical literature, and technical white papers—frequently presents domain-shift challenges. Pre-trained checkpoints reflect the broad lexical patterns of general web and news corpora, often struggling with specialized terminology, dense domain structures, and atypical syntax when unadapted.

Domain adaptation resolves this discrepancy by exposing the pre-trained model to target-domain corpora. This can be accomplished through continued self-supervised pre-training, where the model continues its denoising or masked-language objectives on in-domain documents before task fine-tuning. This process updates the internal attention weights to recognize domain-specific jargon, technical nomenclature, and complex rhetorical patterns without catastrophic forgetting of general syntactic rules.

To reduce the computational burden of fine-tuning multi-million parameter models, Parameter-Efficient Fine-Tuning (PEFT) techniques have become a standard alternative to full-model fine-tuning. Methods such as Low-Rank Adaptation (LoRA) freeze the primary pre-trained weight matrices and inject trainable low-rank decomposition matrices into the multi-head attention projection layers. Other parameter-efficient strategies include prefix tuning, which prepends continuous, task-specific virtual token vectors to the key and value states, and adapter modules, which insert small bottleneck feed-forward layers directly after Transformer attention blocks. These techniques dramatically reduce the number of trainable parameters by up to 99%, allowing models to adapt to new linguistic complexity tiers and specialized domains while minimizing storage footprints and training latency.

Chapter 6: Generative Adversarial Networks (GANs) for Discrete Text

6.1 The Fundamental GAN Formulation (G vs D)

Generative Adversarial Networks (GANs), introduced by Goodfellow et al., frame generative modeling as a minimax zero-sum game between two competing neural networks: a Generator (G) and a Discriminator (D). In traditional continuous domains such as computer vision, the generator maps a continuous noise distribution $p_{\mathrm{z}}(\mathrm{z})$ into synthetic data samples $G(\mathrm{z})$ that emulate real data drawn from a true distribution $p_{\mathrm{data}}(\mathrm{x})$. Simultaneously, the discriminator acts as a binary classifier tasked with calculating the probability that a given sample originated from the true data distribution rather than the generative model.

The minimax optimization objective is formally defined as:

$$\min_{G}\max_{D} V(D,G) = \mathbb{E}_{\mathrm{x}\sim p_{\mathrm{data}}(\mathrm{x})}[\log D\,(\mathrm{x})] + \mathbb{E}_{\mathrm{z}\sim p_{\mathrm{z}}(\mathrm{z})}\big[\log\big(1 - D\big(G(\mathrm{z})\big)\big)\big]$$

During training, D is optimized to maximize the probability of assigning correct categorical labels to both training examples and generated samples:

Conversely, G is trained to fool the discriminator by minimizing $\log\big(1 - D\big(G(\mathrm{z})\big)\big)$, or more commonly in practical implementations, maximizing $\log D\left(G(\mathrm{z})\right)$ to circumvent early gradient saturation:

$$\nabla_{\theta_G}\frac{1}{m}\sum_{i=1}^{m}\log D\left(G\left(\mathrm{z}^{(i)}\right)\right)$$

Under theoretical equilibrium conditions, this adversarial interaction drives the generative distribution p_g to converge toward the true data distribution p_{data}, at which point the discriminator achieves an optimal non-informative classification accuracy $D(\mathrm{x}) = \frac{1}{2}$.

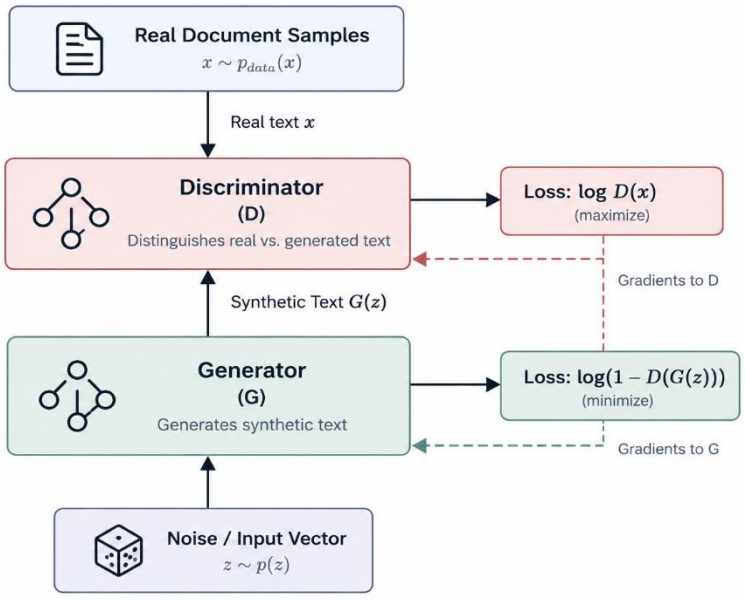


Fig 6.1 Flowchart showing architecture and workflow of GAN

6.2 The Non-Differentiability Bottleneck in Discrete Tokens

While GAN architectures achieve strong empirical success in continuous real-valued domains (e.g., image and audio synthesis), adapting the minimax framework directly to natural language processing encounters a fundamental mathematical obstacle: the discrete token selection bottleneck.

In autoregressive language generation, the generator maintains hidden representations h_t and projects them through a linear layer to emit a categorical probability distribution over the target vocabulary $\mathcal{V}$ via the Softmax function:

$$\mathrm{p}_t = \mathrm{softmax}(\mathrm{W}_v \mathrm{h}_t)$$

To emit actual text strings that can be processed by downstream layers or fed into the discriminator, the network must sample a discrete token $y_t \in \mathcal{V}$ from this probability distribution:

$$y_t = \arg\max_{w \in \mathcal{V}}(\mathrm{p}_t) \quad \text{or} \quad y_t \sim \mathrm{Categorical}(\mathrm{p}_t)$$

The arg max operation and categorical sampling steps are non-continuous step functions whose mathematical derivatives are zero almost everywhere, with infinite discontinuities at decision boundaries:

$$\frac{\partial y_t}{\partial \mathrm{p}_t} = 0 \quad \text{(almost everywhere)}$$

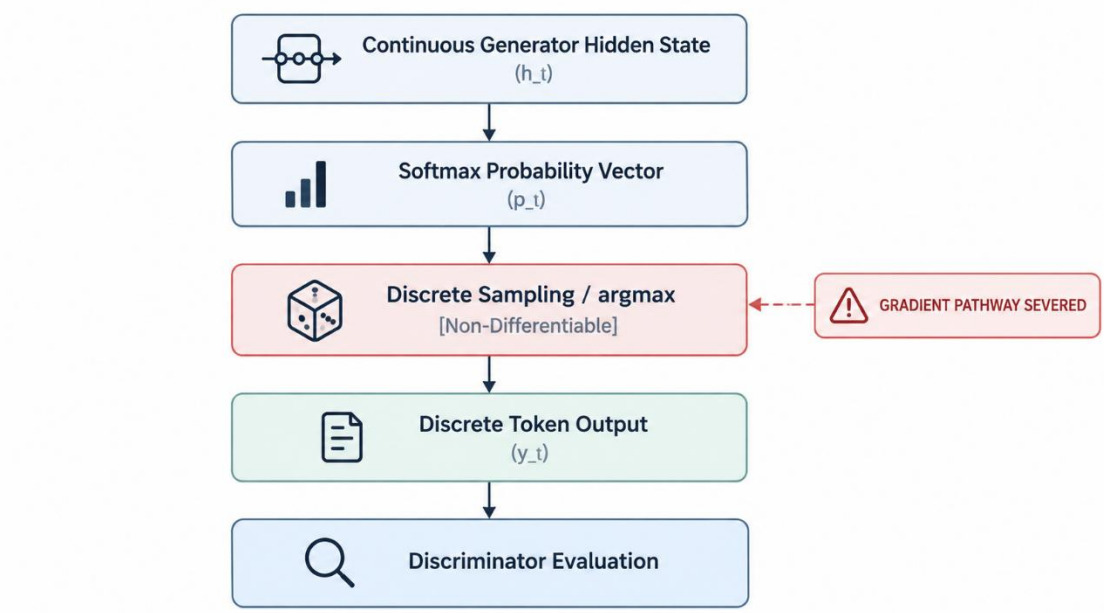


Fig 6.2 Challenges of Backpropagation in Adversarial Text Generation

Because discrete token selection severs the computational graph, backpropagation via the standard chain rule cannot calculate gradients of the discriminator's scalar output with respect to the generator's internal parameters θ_G:

$$\frac{\partial \mathcal{L}_G}{\partial \theta_G} = \frac{\partial \mathcal{L}_G}{\partial D(Y)} \cdot \frac{\partial D(Y)}{\partial Y} \cdot \underbrace{\frac{\partial Y}{\partial \mathbf{p}_t}}_{=0} \cdot \frac{\partial \mathbf{p}_t}{\partial \theta_G}$$

Consequently, small parameter updates in θ_G cannot change the sampled discrete token y_t, leaving the generator without a functional training signal to update its weights through standard gradient descent.

6.3 Reinforcement Learning Solutions: Policy Gradient & REINFORCE

6.3.1 Formulating Text Generation as an MDP

In this formulation:

a) *Agent*: The generative language model (G).
b) *State* (s_t): The sequence of tokens generated up to time step t, denoted as $Y_{1:t-1} = (y_1, y_2, \dots, y_{t-1})$, conditioned on the source text **x**.
c) *Action* (a_t): The selection of the next discrete token y_t chosen from the vocabulary action space $\mathcal{V}$.
d) *Policy* ($\pi_\theta(a_t \mid s_t)$): The conditional probability distribution over the vocabulary emitted by the generator:

$$\pi_\theta(y_t \mid Y_{1:t-1}, \mathbf{x}) = P(y_t \mid Y_{1:t-1}, \mathbf{x}; \theta)$$

e) *Reward* (R): A scalar evaluation signal provided by the discriminator D_ϕ only after the complete sequence $Y_{1:T}$ is generated

6.3.2 The Policy Gradient Theorem and REINFORCE

The goal of the generator is to maximize the expected cumulative reward of the generated sequences under its parameterized policy:

$$J(\theta) = \mathbb{E}_{Y_{1:T} \sim \pi_\theta}[R(Y_{1:T})] = \sum_{Y_{1:T}} P\,(Y_{1:T}; \theta) R(Y_{1:T})$$

Applying the log-derivative trick ($\nabla_\theta P = P \nabla_\theta \log P$), the gradient of the expected reward with respect to the generator parameters θ is derived via the Policy Gradient Theorem:

$$\nabla_\theta J(\theta) = \mathbb{E}_{Y_{1:T} \sim \pi_\theta}\left[\sum_{t=1}^{T} \nabla_\theta \log \pi_\theta\,(y_t \mid Y_{1:t-1}) Q(s_t, y_t)\right]$$

Using the REINFORCE algorithm (Williams, 1992), this expected value is approximated empirically using Monte Carlo sampling over K generated rollouts:

$$\nabla_\theta J(\theta) \approx \frac{1}{K} \sum_{k=1}^{K} \sum_{t=1}^{T} \nabla_\theta \log \pi_\theta \left(y_t^{(k)} \mid Y_{1:t-1}^{(k)}\right) \left(R\left(Y_{1:T}^{(k)}\right) - b(s_t)\right)$$

Where $b(s_t)$ represents a baseline state-value estimate subtracted from the reward to reduce the variance of the gradient estimator without introducing bias.

Component	Standard Supervised Seq2Seq	Policy Gradient RL Formulation
Loss Signal	Token-level Cross-Entropy $(-\log P\,(y_t^*))$	Sequence-level Discriminator Reward $(R(Y_{1:T}))$
Input Supervision	Ground-truth reference tokens	Autoregressively sampled policy rollouts
Exposure Bias	High (Teacher forcing discrepancy)	Low (Trained directly on self-generated paths)
Gradient Flow	Direct backpropagation through Softmax	Expected score-function trick $(\nabla_\theta \log \pi_\theta)$

Table 6.1 Comparison between Standard Supervised Seq2Seq and Policy Gradient RL Formulation

6.4 Actor-Critic Architectures for Natural Language

While the basic REINFORCE algorithm circumvents the non-differentiability bottleneck, it exhibits high gradient variance. Because a reward $R(Y_{1:T})$ is returned only after the complete sequence is synthesized, every token generated in the sequence receives the same global terminal reward, regardless of whether a specific word choice improved or impaired the overall summary quality.

To address this credit assignment challenge, natural language frameworks incorporate Actor-Critic architectures:

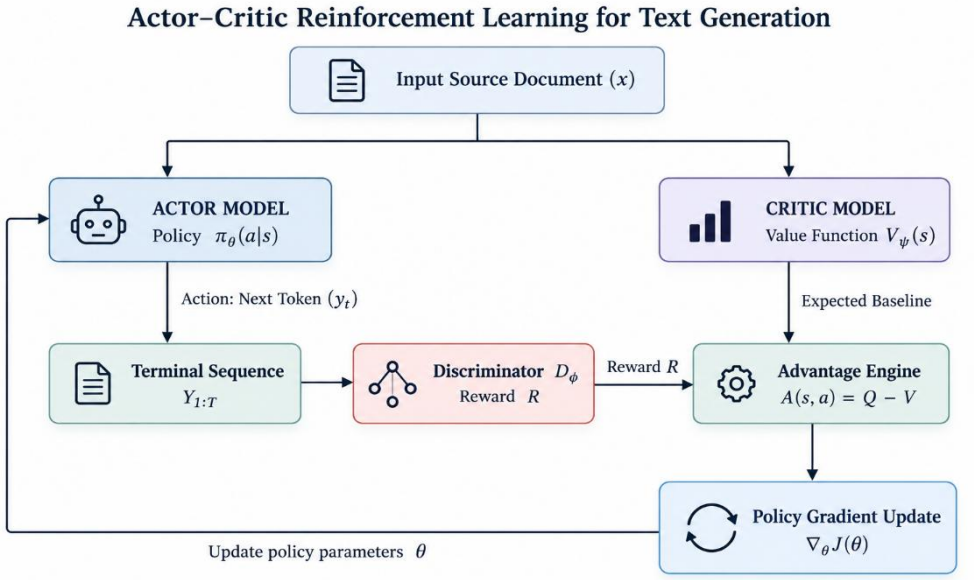


Fig 6.3 Actor-Critic Architecture Workflow Diagram

6.4.1 The Actor Network (Policy Formulation)

The **Actor** is the sequence-to-sequence generator G_θ. It maintains the generative policy $\pi_\theta(y_t \mid Y_{1:t-1})$ and is responsible for selecting discrete actions (tokens) from the vocabulary.

6.4.2 The Critic Network (Value Estimation)

The "Critic" is an auxiliary neural network $V_\psi(s_t)$ parameterized by ψ. It evaluates intermediate generation states, predicting the expected cumulative reward from the current partial sequence $Y_{1:t}$ until sequence completion:

$$V_\psi(s_t) \approx \mathbb{E}[R(Y_{1:T}) \mid Y_{1:t}]$$

6.4.3 Advantage Function and Temporal Difference Updates

Instead of weighting policy updates with raw terminal rewards, the Actor is updated using the **Advantage Function** $A(s_t, y_t)$, which measures how much better a chosen token is compared to the expected baseline value of that state:

$$A(s_t, y_t) = Q(s_t, y_t) - V_\psi(s_t)$$

Using a 1-step Temporal Difference (TD) target, the intermediate action-value $Q(s_t, y_t)$ is approximated as:

$$Q(s_t, y_t) \approx r_t + \gamma V_\psi(s_{t+1})$$

The Actor and Critic parameters are updated through alternating optimization:

- **Critic Update:** Minimize the Mean Squared Error (MSE) between the predicted state values and the observed TD targets:

$$\mathcal{L}(\psi) = \frac{1}{T}\sum_{t=1}^{T}\left(r_t + \gamma V_\psi(s_{t+1}) - V_\psi(s_t)\right)^2$$

- **Actor Update:** Scale the log-probability of the policy by the calculated advantage estimate:

$$\nabla_\theta \mathcal{L}(\theta) = \frac{1}{T}\sum_{t=1}^{T} \nabla_\theta \log \pi_\theta\,(y_t \mid s_t) A(s_t, y_t)$$

By balancing sequence-level evaluation with fine-grained step-wise baseline estimation, Actor-Critic architectures provide stable gradient updates, resolve the discrete credit assignment problem, and guide text generation models toward high factual consistency and target complexity standards.

Chapter 7: The Reado.ai Tri-Level Contextual Text GAN

7.1 System Architecture of the Reado.ai Engine